Construcción Formal de Programas en Teoría de Tipos
Primer Parcial – 2026

Problema 1. Considere las siguientes reglas de un sistema de verificación de software:

1. *Si tienes la clave de cifrado, el archivo se desbloquea*
2. *El archivo no se desbloquea o los datos se guardan*
3. *Es imposible tener la clave de cifrado y que ocurra un error de sistema a la vez*

A partir de estas reglas, pruebe las siguientes sentencias, sin usar tácticas automáticas ni lógica clásica:

- *Si hay un error de sistema entonces no se tiene la clave de cifrado*
- *Los datos se guardan o no tienes la clave de cifrado*

Problema 2. ¿Es posible demostrar en Rocq (Coq): $\forall$x:D. P(x) $\rightarrow$ $\exists$z:D. P(z), dada una propiedad P sobre elementos de un dominio genérico D? En caso afirmativo, formalice y realice la prueba. De lo contrario explique por qué no es posible.

Problema 3. Imagina que estamos verificando las dependencias en un sistema operativo. Tenemos: i) un tipo de datos para los procesos (Proceso); ii) Una relación Asignado x y: el proceso x tiene asignado el recurso y; iii) Una relación Bloqueado x y: El proceso x está esperando/bloqueado por el proceso y. Queremos demostrar que si se cumplen dos condiciones del sistema, entonces existe un proceso que está en una situación crítica (tiene un recurso asignado y a la vez está bloqueado por alguien).

```
Variable Proceso Recurso : Set.
Variable Asignado : Proceso -> Recurso -> Prop.
Variable Bloqueado : Proceso -> Proceso -> Prop.
```

Demuestre sin usar tácticas automáticas ni lógica clásica:

```
Theorem t3 :
 forall p_inicio : Proceso,
 (exists r : Recurso, forall p : Proceso, Asignado p r) ->
 (forall p1 : Proceso, exists p2 : Proceso, Bloqueado p1 p2) ->
 exists x y : Proceso, exists w : Recurso, Bloqueado x y /\ Asignado x w.
```

Problema 4. Considere las siguientes declaraciones:

```
Variable Bool: Set.
Variable TRUE FALSE : Bool.
Variable Not : Bool -> Bool.
Variable Imp Xor: Bool -> Bool -> Bool.

Axiom Disc : ~ (FALSE = TRUE).
Axiom BoolVal : forall b : Bool, b = TRUE \/ b = FALSE.
Axiom NotTrue : Not TRUE = FALSE.
Axiom NotFalse : Not FALSE = TRUE.
Axiom ImpFalse : forall b : Bool, Imp FALSE b = TRUE.
Axiom ImpTrue : forall b : Bool, Imp TRUE b = b.
Axiom XorTrue : forall b : Bool, Xor TRUE b = Not b.
Axiom XorFalse : forall b : Bool, Xor FALSE b = b.
```

Bajo el contexto previo, demuestre en Rocq (Coq) los siguientes lemas:

a) *Lemma l4_1: forall b1 b2: Bool, Imp b1 b2 = FALSE -> b1 = TRUE /\ b2 = FALSE.*
b) *Lemma l4_2: forall b1 b2: Bool, Xor b1 b2 = TRUE -> Imp b1 (Not b2) = TRUE.*

Problema 5. Considere el constructor de tipos `ArbNat` de árboles n-arios de números naturales:

```
Parameter ArbNat : forall n : nat, Set.
```

Recuerde que un árbol n-ario puede ser vacío ó consiste de un nodo con un dato de tipo nat (en este caso) y tiene exactamente n subárboles n-arios.

1. Defina el tipo del operador `null` que permita representar un árbol n-ario vacío (sin nodos).
2. Defina el tipo del operador `add` que permita generar árboles n-arios no vacíos de naturales. Este operador junto con `null` deberían permitir crear cualquier árbol n-ario de naturales.
3. Construya utilizando los operadores `null` y `add` un árbol 3-ario no vacío de naturales que tenga un solo nodo con el valor 8 como raíz.